

```

# 8 Do any "per frame" actions
# Check if the ball is colliding with the target
❷ ballRect = pygame.Rect(ballX, ballY,
                           BALL_WIDTH_HEIGHT, BALL_WIDTH_HEIGHT)
❸ if ballRect.colliderect(targetRect):
    print('Ball is touching the target')

# 9 - Clear the window
window.fill(BLACK)

# 10 - Draw all window elements
❹ window.blit(targetImage, (TARGET_X, TARGET_Y)) # draw the target
window.blit(ballImage, (ballX, ballY)) # draw the ball

# 11 - Update the window
pygame.display.update()

# 12 - Slow things down a bit
clock.tick(FRAMES_PER_SECOND) # make pygame wait

```

Listing 5-4: Detecting and acting on single key presses

First we add a few new constants ❶ to define the x- and y-coordinates of the upper-left corner of the target rectangle and the width and height of the target. We then load the image of the target rectangle ❷.

In the loop where we look for for events, we add a test for a key press by checking for an event of type `pygame.KEYDOWN` ❸. If a key down event is detected, we look into the event to find out what key was pressed. Each key has an associated constant in pygame, so here we check if the user has pressed the left, up, down, or right arrow. For each of these keys, we modify the value of the ball's x- or y-coordinate appropriately by a small number of pixels.

Next we create a pygame `rect` object for the ball based on its x- and y-coordinates and its height and width ❹. We can check to see if two rectangles overlap with the following call:

```
<booleanVariable> = <rect1>.colliderect(<rect2>)
```
